

MANUAL DE FLUJO DE TRABAJO

con Git y GitHub

Guía para quien nunca ha trabajado con Git/GitHub en equipo
Versión genérica — aplicable a cualquier proyecto

Antes de usar este manual: Este documento explica el flujo general. Cada proyecto puede ajustar detalles puntuales (nombre del repositorio, alcances de commit, canal de dudas). Si algo no calza con tu proyecto o tienes dudas, consúltalo con la persona que te compartió este manual antes de improvisar.

0. Antes de empezar

Si nunca has usado Git ni GitHub en equipo, no te preocupes. Este manual explica el porqué, el qué y el cómo, paso a paso, con los comandos exactos. La idea es que sigas siempre el mismo camino para que el trabajo de todo el equipo se pueda integrar sin caos.

0.1 Conceptos básicos (léelos una vez y vuelve cuando lo necesites)

- Git: programa en tu computadora que guarda el historial de cambios del código. Cada «foto» del código se llama commit.
- GitHub: sitio web donde vive una copia del repositorio. Ahí se revisan los cambios (Pull Requests) y se integran a la rama principal.
- Repositorio (repo): la carpeta del proyecto con todo su historial.
- Rama (branch): una línea de trabajo paralela. main (o master) es la oficial y siempre debe quedar estable. Tú trabajas en tu propia rama para no romperla.
- Commit: un paquete de cambios con un mensaje que explica qué hiciste.
- Pull Request (PR): solicitud en GitHub para que tu rama se integre a main. Alguien más la revisa y la aprueba antes de mergear.
- Issue / ticket: tarjeta de trabajo en GitHub (por ejemplo #18, #63). Cada rama y cada commit se vinculan a un número de issue.
- Merge: acción de unir los commits de una rama dentro de otra (normalmente de tu rama hacia main).

0.2 Cómo se organiza el equipo en la práctica

- Cada persona toma un ticket y trabaja en su propia rama.
- Se implementa el cambio, se hacen commits atómicos (pequeños y con sentido) y se abre un Pull Request.
- Alguien más del equipo revisa el PR. Si encuentra defectos, se corrigen en la misma rama.
- Cuando el PR se aprueba, se mergea a main. Después cada quien actualiza su main local.
- Si varios tickets dependen entre sí, se coordina con el resto del equipo antes de mezclar ramas por tu cuenta.

1. Nomenclatura de ramas

Toda rama de trabajo se crea a partir de main actualizado y sigue este formato:

`tipo/numero-issue-descripcion-corta`

Todo en minúsculas, palabras separadas por guiones, sin espacios ni acentos en el nombre de la rama.

1.1 Tipos de rama

Prefijo	Cuándo usarlo	Ejemplo de formato
feat/	Nueva funcionalidad (la mayoría del trabajo)	feat/12-nombre-corto
fix/	Corrección de un bug (a menudo tras revisión de un PR)	fix/12-nombre-corto
docs/	Solo documentación	docs/12-nombre-corto
chore/	Mantenimiento, configuración, tareas que no son feature ni bug	chore/9-nombre-corto
refactor/	Reestructurar código sin cambiar su comportamiento	refactor/5-nombre-corto
integracion/	Unir varias ramas que dependen entre sí antes de entrar a main	integracion/nombre-descriptivo

Regla de oro: Una rama = un tiquete (un issue). No mezcles dos issues en la misma rama. Si el tiquete es grande, los commits internos lo desglosan, pero el PR cierra un solo número.

2. Convención de commits (Conventional Commits)

Los mensajes se escriben en minúsculas (salvo nombres propios o códigos), en tiempo presente o imperativo, e incluyen siempre el número de issue.

2.1 Estructura del mensaje

tipo(alcance): descripción corta en minúsculas (#número)

- Viñeta 1 con detalle de lo que cambió.
- Viñeta 2 con detalle de lo que cambió.
- ...
- ...

Closes #número

2.2 Tipos más comunes

Tipo	Cuándo usarlo
feat	Agrega una funcionalidad nueva
fix	Corrige un error
docs	Cambios solo de documentación
chore	Mantenimiento, configuración, dependencias
refactor	Reestructura código sin cambiar el comportamiento

El alcance (lo que va entre paréntesis) depende de las capas de tu proyecto: puede ser *api*, *web*, *db*, *móvil*, *ui*, o el nombre de un módulo. Cada equipo puede definir sus propios alcances.

2.3 Commit de «arranque» (buena práctica opcional)

Antes de escribir código, muchos equipos hacen un commit solo de documentación que anota: qué se va a hacer, qué decisiones se tomaron, y qué riesgos o dudas hay. Esto evita reescribir código a mitad del tiquete y deja rastro para quien revisa.

docs: arranque del tiquete #18 (descripción breve)

3. Flujo de trabajo principal (paso a paso)

Este es el camino que debes seguir cada vez que tomas un ticket. No te saltes pasos.

Paso 1 — Sincronizar main

Siempre parte de la versión más reciente. Si no lo haces, tu rama nacerá desfasada y el PR generará conflictos.

```
git checkout main
git pull origin main
```

Paso 2 — Crear tu rama

Usa el número del issue y una descripción corta:

```
git checkout -b feat/18-descripcion-corta
```

Paso 3 — (Recomendado opcional) Commit de arranque

```
git add docs/...
git commit -m "docs: arranque del ticket #18 (descripcion breve)"
```

Paso 4 — Desarrollar y hacer commits atómicos

Trabaja en ciclos cortos. Cada commit debe dejar el proyecto en un estado coherente.

```
git status          # ver qué cambió
git add .           # o git add ruta/archivo
git commit -m "feat(alcance): descripcion breve (#18)"

- Detalle 1 de lo que se hizo.
- Detalle 2 de lo que se hizo.
- ...
- ...

closes: #18"
```

Consejo: Si el mensaje es largo, usa `git commit` (sin `-m`) y se abrirá el editor para escribir título + cuerpo.

Paso 5 — Publicar la rama y abrir el Pull Request

```
git push -u origin feat/18-descripcion-corta
```

La primera vez usa `-u` (upstream) para vincular tu rama local con la remota. Las siguientes veces basta con `git push`.

En GitHub:

- Entra al repositorio; verás un banner «Compare & pull request».
- Título del PR: suele ser el mismo del commit principal.
- Descripción: resume qué se hizo, decisiones tomadas, cómo probar y qué queda fuera de alcance.
- Solicita revisión a un compañero o a quien coordine el repositorio.
- Si el ticket cierra el issue, escribe Closes #18 en la descripción del PR.

Paso 6 — Atender la revisión que hizo quién coordina

- Corriges los defectos en la misma rama (no crees otra).
- Haces commits fix(alcance): ... (#número) explicando causa y verificación.
- Haces git push (la rama ya está vinculada).
- Respondes en el hilo del PR cuando quede listo para volver a revisar.

Paso 7 — Después del merge: limpiar en local

```
git checkout main
git pull origin main
git fetch --prune
git branch -d feat/18-descripcion-corta
```

fetch --prune limpia las referencias a ramas remotas ya borradas. branch -d borra tu rama local de forma segura (falla si no está integrada).

4. Diagrama del flujo (vista rápida)

```
[ main local ]
| git pull origin main
crear rama tipo/XX-descripcion
| (opcional) docs: arranque del tiquete #XX
commits feat/fix mientras desarrollas
| git push origin tipo/XX-descripcion
abrir Pull Request en GitHub
| revisión -> commits fix si hace falta
Merge del PR en main (en GitHub)
| git checkout main && git pull
git branch -d tipo/XX-descripcion
```

5. Situaciones frecuentes (y cómo resolverlas)

5.1 main avanzó mientras trabajabas

Otro compañero ya mergeó su PR. Necesitas traer esos cambios a tu rama:

```
git checkout main
git pull origin main
git checkout feat/18-descripcion-corta
git merge main
```

Si hay conflictos, Git los marca en los archivos. Ábrelos, busca las marcas <<<<<<< ===== >>>>>>>, deja el código correcto, luego:

```
git add .
git commit # cierra el merge; el mensaje lo propone Git
```

Si varios tiquetes dependen entre sí: y se crearon encadenados uno sobre otro (en vez de partir de main), puede ser necesario usar una rama integracion/... para unirlos antes de entrar con un solo PR. Si te pasa, coordina con el resto del equipo antes de forzar merges.

5.2 Necesitas pausar sin hacer commit

```
git stash          # guarda cambios temporalmente
git checkout main  # o la rama que necesites
# ... trabajas en otra cosa ...
git checkout feat/18-descripcion-corta
git stash pop      # recupera tus cambios
```

5.3 Descartar cambios locales no deseados

Un solo archivo:

```
git checkout -- ruta/del/archivo
```

Todos los cambios no guardados en staging:

```
git restore .
```

Cuidado: Esto borra trabajo no commiteado. No hay «deshacer» fácil después.

5.4 Te equivocaste en el último commit (aún no pusheado)

Corregir el mensaje:

```
git commit --amend -m "feat(alcance): mensaje corregido (#18)"
```

Agregar un archivo que olvidaste:

```
git add archivo-olvidado
git commit --amend --no-edit
```

Importante: Solo usa --amend si aún NO has hecho push. Si ya está en GitHub, no reescribas historia: haz un commit nuevo.

5.5 Conflictos en un PR

GitHub te avisará. En local:

```
git checkout feat/tu-rama
git fetch origin
git merge origin/main
# resuelves conflictos
git add .
git commit
git push
```

6. Buenas prácticas generales

- Committea en unidades lógicas pequeñas, no un solo commit gigante al final.
- Nunca trabajes directo en main: todo cambio entra por Pull Request.
- Prueba tu cambio antes de pedir revisión (camino feliz y casos de error).
- Sé claro en los mensajes de commit y en la descripción del PR: qué y por qué, no solo “cambios”.
- No hagas push --force a main ni a una rama compartida salvo indicación explícita.
- Si algo del proyecto no está claro (una regla de negocio, un dato, un permiso), pregunta antes de improvisar.
- Si tu tiquete toca una tabla, endpoint o pantalla que ya usa otra parte del sistema, evita duplicar lógica: reutiliza lo que ya existe.
- Si algún compañero del equipo está trabajando en un archivo que debes trabajar, comunícate con él para evitar conflictos en el repositorio.

7. Glosario de comandos útiles

Comando	Para qué sirve
git status	Ver archivos modificados, en staging y no rastreados
git log --oneline -n 15	Últimos 15 commits en una línea
git log -n 3	Últimos 3 commits con mensaje completo
git diff	Ver cambios aún no agregados al staging
git diff --staged	Ver lo que ya está en staging (listo para commit)
git fetch --prune	Actualizar referencias remotas y borrar las de ramas ya eliminadas
git branch	Listar ramas locales (* marca la actual)
git branch -d nombre	Borrar rama local ya integrada
git remote -v	Ver la URL del remoto (origin)
git stash / git stash pop	Guardar y recuperar cambios temporales

8. Checklist antes de abrir un Pull Request

- Partí de main actualizado (git pull) antes de crear la rama.
- El nombre de la rama es tipo/numero-descripcion-corta.
- Cada commit sigue Conventional Commits e incluye (#número).
- El cuerpo del commit (o del PR) explica decisiones, no solo «se hizo el feature».
- Probé el camino feliz y los casos de error.
- Si agregué algo nuevo (permiso, campo, endpoint), lo dejé documentado.
- La descripción del PR permite a otra persona probar sin tener que preguntarme.
- Incluí Closes #XX si el PR termina el tiquete.

9. Errores comunes de quien empieza (y cómo evitarlos)

- Trabajar directo en main. Nunca. main solo recibe cambios por Pull Request.
- Crear la rama sin hacer pull. Empiezas desfasado; el PR chocará con main.
- Un solo commit gigante al final. Pierdes historial útil y complica la revisión.
- Mensajes vagos (“cambios”, “fix”, “update”). El mensaje debe decir qué y por qué, con el número de issue.
- Pushar --force a main o a una rama compartida. Puede borrar trabajo de otros.
- Ignorar la revisión. Si alguien señaló un defecto, respóndelo y corrígelo en la misma rama.
- Cerrar el issue a mano. Usa Closes #XX en el PR o en el commit final; GitHub lo cierra al merge.

10. Tu primer día en el repo (ruta sugerida)

1. Instala Git y configura tu nombre y correo (los mismos de GitHub):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
```

2. Clona el repositorio (URL que te compartan):

```
git clone <url-del-repositorio>
cd <nombre-del-repositorio>
```

3. Verifica que estás en main y actualizado:

```
git checkout main
git pull origin main
git log --oneline -n 10
```

4. Lee el issue asignado en GitHub (criterios de aceptación, mockups, comentarios).

5. Crea tu rama feat/... y, si aplica, el commit de arranque con tus hallazgos.

6. Desarrolla, commitea, pushea y abre el PR siguiendo las secciones 3 y 8 de este manual.

11. Resumen en una frase

Trabaja siempre en una rama ligada a un issue, escribe commits claros con el número del tiquete, documenta las decisiones difíciles, abre un Pull Request para revisión, corrige en la misma rama y solo después integra a main.

--- Fin del documento ---

Cualquier duda sobre este flujo o sobre el proyecto: consúltala con la persona que te compartió este manual antes de improvisar cambios en main.